SHOPSMART — DSA VIVA EXAMINATION STUDY GUIDE & QUESTION BANK

Comprehensive Viva Voce Preparation for Data Structures & Algorithms (C + SQLite + Web)

TABLE OF CONTENTS

1. PROJECT OVERVIEW & ARCHITECTURAL OVERVIEW
2. BINARY SEARCH TREE (BST) — PRODUCT SEARCH & INDEXING
3. HASH TABLE WITH SEPARATE CHAINING — CONSTANT-TIME LOOKUPS
4. SINGLY LINKED LIST — INTERACTION HISTORY TRACKING
5. MAX HEAP (PRIORITY QUEUE) — TOP-K RECOMMENDATION RANKING
6. HYBRID RECOMMENDATION MATHEMATICAL MODEL
7. TIME & SPACE COMPLEXITY CHEAT SHEET
8. TRICKY EXAMINER QUESTIONS & DEFENSE STRATEGIES

1. PROJECT OVERVIEW & ARCHITECTURE

Q1. What is the ShopSmart project, and what role do Data Structures play in it?

Answer:
ShopSmart is an e-commerce platform developed with a high-performance **C backend** and a modern **React web frontend**, backed by **SQLite**.
Rather than treating Data Structures as purely theoretical concepts, ShopSmart embeds four classic data structures directly into core e-commerce workflows:

1. **Binary Search Tree (BST)**: Performs lexicographical catalogue indexing and fast string searches by product title.
2. **Hash Table (Separate Chaining)**: Delivers $O(1)$O(1) constant-time product retrieval by unique `productId` for cart updates and price computations.
3. **Singly Linked List**: Dynamically records chronological user interaction streams (Product Views, Clicks, Wishlists, Add-to-Carts, and Purchases).

4. **Max Heap (Priority Queue)**: Extracts the top KK personalized product recommendations based on a multi-objective recommendation score.

Q2. Why did you choose C for implementing the data structures instead of relying on high-level Python or JavaScript libraries?

Answer:
Implementing the data structures in C demonstrates fundamental computer science competency:

- **Direct Memory Management**: We explicitly manage dynamic memory allocations using `malloc()`, `calloc()`, and deallocate using `free()`, avoiding garbage-collection overhead.
- **Cache Locality and Pointer Semantics**: We manipulate nodes and bucket pointers directly, demonstrating deep understanding of pointer traversal and memory layout.
- **Microsecond Latency**: The compiled C binary evaluates recommendation heaps and tree searches orders of magnitude faster than interpreted runtimes.

2. BINARY SEARCH TREE (BST)

Q3. Where and why is the Binary Search Tree used in ShopSmart?

Answer:
The BST is defined in `backend/include/BST.h` and implemented in `backend/src/BST.c`. It is used to index product names lexicographically (alphabetically).
When a user searches for a product title (e.g., `"Akuma Strike Heavyweight Hoodie"`), the search query traverses the BST instead of scanning an unsorted array sequentially.

Q4. How does the BST compare product names during insertion and search?

Answer:
To prevent case-sensitivity issues, our implementation uses a helper function `normalizeName()` that transforms characters to lowercase via `tolower()`. It then invokes standard `strcmp()`:

```
c
int comparison = strcmp(searchName, rootName);
if (comparison == 0) return &root->product;
if (comparison < 0)  return searchNode(root->left, name);
else                 return searchNode(root->right, name);
```

- If `strcmp < 0`: Traverse to the **left child**.

- If `strcmp > 0`: Traverse to the **right child**.
- If `strcmp == 0`: Exact match found.

Q5. What are the Time and Space Complexities of BST operations in ShopSmart?

Answer:

- **Search Time Complexity**:
  - **Average / Best Case**: $O(\log N)$O(log N) when the tree is reasonably balanced.
  - **Worst Case**: $O(N)$O(N) if elements are inserted in already-sorted order, turning the tree into a skewed degenerate linked list.
- **Insertion Time Complexity**: Average $O(\log N)$O(log N), Worst $O(N)$O(N).
- **Space Complexity**: $O(N)$O(N) total space for NN nodes; $O(h)$O(h) auxiliary stack space during recursive traversal, where hh is the tree height ($h = \log N$h = log N average, $h = N$h = N worst-case).

Q6. How do you delete a node in a Binary Search Tree?

Answer:
Deleting a node involves three cases:

1. **Node is a leaf (no children)**: Simply free the node and return `NULL`.
2. **Node has one child**: Replace the node with its child, free the deleted node, and reconnect the parent pointer.
3. **Node has two children**:
   - Find the **In-Order Successor** (the smallest value in the right subtree) using `findMinimum(root->right)`.
   - Copy the successor's product data into the current node.
   - Recursively delete the successor from the right subtree.

3. HASH TABLE WITH SEPARATE CHAINING

Q7. Where is the Hash Table used in ShopSmart?

Answer:
The Hash Table is declared in `backend/include/HashTable.h` and implemented in `backend/src/HashTable.c`. It stores and indexes products by their integer `productId`.

When a user adds an item to cart or initiates checkout, the server must look up product prices, inventory, and details by `productId` in $O(1)$O(1) constant time.

Q8. Which Hash Function and Collision Resolution Technique do you use?

Answer:

- **Hash Function**: Division Method (Modulo Arithmetic): $\text{Index} = \text{key} \pmod{\text{tableSize}}$ Index = key (mod tableSize) In C: `(size_t)key % table->tableSize;`
- **Collision Resolution**: **Separate Chaining**. Each bucket in `table->buckets` is a pointer to the head of a singly linked list of `HashEntry` nodes:

  ```
  c
  typedef struct HashEntry {
      int key;
      Product product;
      struct HashEntry* next;
  } HashEntry;
  ```

  If two product IDs produce the same bucket index, the new entry is prepended to that bucket's linked list.

Q9. What is Load Factor (αα), and how does your Hash Table handle rehashing?

Answer:
The load factor is the ratio of stored elements to total bucket capacity: $\alpha = \frac{\text{elementCount}}{\text{tableSize}}$ α = tableSizeelementCountIn `backend/src/HashTable.c`:

- When αα grows, bucket chains become longer, degrading search performance from $O(1)$O(1) toward $O(N)$O(N).
- Our implementation triggers dynamic **Rehashing** (`rehash()`):
  1. Doubles `tableSize` ($2 \times \text{tableSize}$2 × tableSize).
  2. Allocates a new bucket array using `calloc()`.
  3. Re-hashes every existing entry into the new larger bucket array.
  4. Frees the old bucket pointer table. This guarantees that average lookup, insertion, and deletion remain strictly $O(1)$O(1).

4. SINGLY LINKED LIST

Q10. Why is a Singly Linked List used for User Interaction History?

Answer:
Defined in `backend/include/LinkedList.h` and `backend/src/LinkedList.c`, the Linked List records an unbounded chronological sequence of user actions:

```
c
typedef struct InteractionNode {
    int productId;
    char interactionType[MAX_INTERACTION_TYPE]; /* "View", "Wishlist", "Add to Cart", etc.
*/
    char timestamp[MAX_TIMESTAMP];
    struct InteractionNode* next;
} InteractionNode;
```

Why a Linked List over a Fixed Array?

1. **Dynamic Size**: Users can perform an unpredictable number of browsing actions. An array would require continuous reallocations and memory copying.
2. **Efficient Append**: Appending an interaction to the history list takes $O(1)$O(1) when tracking the tail pointer, with zero risk of buffer overflow.
3. **Sequential Processing**: When generating recommendations, we need sequential pass-through of recent interactions to weight user category preferences.

Q11. How does the Linked List assist the Recommendation Engine?

Answer:
In `RecommendationEngine.c`, the function `historyContainsProduct(LinkedList* history, int productId)` traverses the list to filter out merchandise the user has already interacted with or purchased, ensuring recommendations surface novel items.

5. MAX HEAP (PRIORITY QUEUE)

Q12. What is a Max Heap, and how is it used in ShopSmart?

Answer:
Defined in `backend/include/MaxHeap.h` and implemented in `backend/src/MaxHeap.c`, a Max Heap is a

complete binary tree satisfying the **Max Heap Property**:

> *The value/score of every parent node is greater than or equal to the values of its children.*

In ShopSmart, each heap node stores a candidate product along with its calculated recommendation score:

```
c
typedef struct {
    Product product;
    double score;
} RecommendationItem;
```

The root of the heap always holds the product with the **highest recommendation score**.

Q13. How is the Max Heap represented in memory?

Answer:
Rather than using node pointers, the complete binary tree is mapped into a contiguous 1D array (`RecommendationItem items[]`). For any node at index ii:

- **Parent Index**: $\lfloor \frac{i-1}{2} \rfloor$⌊2i−1⌋
- **Left Child Index**: $2i + 1$2i + 1
- **Right Child Index**: $2i + 2$2i + 2

This array representation delivers superior CPU cache performance with zero pointer overhead.

Q14. Explain `heapifyUp` and `heapifyDown`.

Answer:

- `heapifyUp(MaxHeap* heap, size_t index)` :
  Used during **Insertion** (`maxHeapInsert`). When a new product and score are added at the end of the array (`heap->items[heap->size++]`), it may violate the heap property. We compare the new item with its parent; if its score is greater, we swap them and repeat upward until the heap property is restored. Time complexity: $O(\log N)$O(log N).
- `heapifyDown(MaxHeap* heap, size_t index)` :
  Used during **Extraction** (`maxHeapExtractMax`). When the top recommendation (root) is removed, we place the last element of the array into index 0 and decrement the size. We compare this element with its left and right children, swap with the largest child if smaller, and repeat downward. Time complexity: $O(\log N)$O(log N).

Q15. Why use a Max Heap instead of simply sorting the entire product array using QuickSort?

Answer:

- If an e-commerce catalogue has NN items (e.g., 100,000) and we only need to display the **Top** K **K recommendations** (e.g., $K = 5$K = 5 or $K = 10$K = 10):
  - **Full Sorting (QuickSort / MergeSort)**: Requires $O(N \log N)$O(N log N) operations to sort every single item in the catalogue.
  - **Max Heap**: Inserting NN items takes $O(N \log N)$O(N log N) (or $O(N)$O(N) with bottom-up heapify), and extracting the top KK items takes only: $O(K \log N)$O(K log N)
- When $K \ll N$K ≪ N, extracting top items from a Heap is significantly faster than sorting the entire dataset.

6. RECOMMENDATION SCORING MATHEMATICS

Q16. What mathematical formula computes the final recommendation score?

Answer:
Implemented in `backend/src/RecommendationEngine.c` (`calculateFinalScore`):
$\text{FinalScore} = 0.60 \times S_{\text{content}} + 0.25 \times S_{\text{preference}} + 0.10 \times S_{\text{popularity}} + 0.05 \times S_{\text{rating}}$ FinalScore = 0.60 × Scontent + 0.25 × Spreference + 0.10 × Spopularity + 0.05 × Srating

1. **Content Similarity Score (S_{content} Scontent)**:
   $S_{\text{content}} = 0.40 \cdot M_{\text{category}} + 0.25 \cdot M_{\text{tags}} + 0.15 \cdot M_{\text{brand}} + 0.10 \cdot P_{\text{sim}} + 0.10 \cdot \frac{\text{popularity}}{100}$Scontent = 0.40 · Mcategory + 0.25 · Mtags + 0.15 · Mbrand + 0.10 · Psim + 0.10 · 100popularityWhere price similarity is: $P_{\text{sim}} = 1.0 - \frac{|P_{\text{user}} - P_{\text{cand}}|}{P_{\text{user}}}$Psim = 1.0 − Puser∣Puser−Pcand∣
2. **User Interaction Weight ($S_{\text{preference}}$ Spreference)**:
   Actions in the Linked List are assigned specific behavioural coefficients:
   - **Product View**: Weight = $1.0$1.0
   - **Search Click**: Weight = $2.0$2.0
   - **Wishlist**: Weight = $4.0$4.0
   - **Add to Cart**: Weight = $5.0$5.0
   - **Purchase**: Weight = $8.0$8.0

7. COMPLEXITY ANALYSIS SUMMARY TABLE

Data Structure	Operation	Best Case	Average Case	Worst Case	Space Complexity
Binary Search Tree	Search (`bstSearch`)	$O(1)$O(1)	$O(\log N)$ O(log N)	$O(N)$O(N) *(skewed)*	$O(N)$O(N)
Binary Search Tree	Insert (`bstInsert`)	$O(1)$O(1)	$O(\log N)$ O(log N)	$O(N)$O(N)	$O(N)$O(N)
Binary Search Tree	In-Order Traversal	$O(N)$O(N)	$O(N)$O(N)	$O(N)$O(N)	$O(\log N)$O(log N) aux stack
Hash Table	Insert (`hashTableInsert`)	$O(1)$O(1)	$O(1)$O(1)	$O(N)$O(N) *(all collide)*	$O(N + M)$ O(N + M)
Hash Table	Search (`hashTableFind`)	$O(1)$O(1)	$O(1)$O(1)	$O(N)$O(N)	$O(N + M)$ O(N + M)
Hash Table	Delete (`hashTableDelete`)	$O(1)$O(1)	$O(1)$O(1)	$O(N)$O(N)	$O(N + M)$ O(N + M)
Linked List	Prepend / Append	$O(1)$O(1)	$O(1)$O(1)	$O(1)$O(1) *(with tail)*	$O(N)$O(N)
Linked List	Search / Contains	$O(1)$O(1)	$O(N)$O(N)	$O(N)$O(N)	$O(1)$O(1) aux
Max Heap	Find Maximum	$O(1)$O(1)	$O(1)$O(1)	$O(1)$O(1)	$O(N)$O(N) array
Max Heap	Insert (`maxHeapInsert`)	$O(1)$O(1)	$O(\log N)$ O(log N)	$O(\log N)$ O(log N)	$O(1)$O(1) aux
Max Heap	Extract Max (`maxHeapExtractMax`)	$O(\log N)$ O(log N)	$O(\log N)$ O(log N)	$O(\log N)$ O(log N)	$O(1)$O(1) aux

8. EXAMINER TRAP QUESTIONS & DEFENSE

Q17. "If a BST can degrade to $O(N)$O(N) in the worst case, why didn't you use an AVL Tree or Red-Black Tree?"

Strong Defense:

> *"In a production environment where dataset order is non-random or adversary-controlled, a Self-Balancing Tree (like an AVL or Red-Black Tree) is preferred to enforce strict $O(\log N)$ height. However, for our catalog size (40–100 items), product insertions occur in non-sorted sequence during CSV load, keeping the BST naturally bushy. Standard BST was deliberately chosen to maintain minimal code footprint and low rotation overhead during catalog loading."*

Q18. "Why not use Open Addressing (Linear/Quadratic Probing) for your Hash Table instead of Separate Chaining?"

Strong Defense:

> *"Separate Chaining was chosen for three reasons: 1. **Simplicity and Reliability during Deletions**: Deleting from open-addressing tables requires tombstone markers to avoid breaking lookup chains. Separate chaining allows clean node freeing. 2. **Graceful Performance Degradation**: If the load factor temporarily spikes above 1.0, separate chaining continues functioning cleanly, whereas open addressing fails once buckets are full. 3. **Non-contiguous Growth**: Chaining avoids primary and secondary clustering phenomena."*

Q19. "How does the React frontend interact with the C data structures?"

Strong Defense:

> *"The C backend runs a REST API server using `libmicrohttpd`. The React client issues asynchronous `fetch()` requests (e.g., `GET /api/recommendations?userId=1`). The C backend queries SQLite for user history, feeds the history into the Linked List, scores candidate products, pushes them into the Max Heap, extracts the top recommendations, serializes them to JSON, and sends the response back to React."*

Q20. "What happens if a user is completely new and has zero interaction history (Cold Start Problem)?"

Strong Defense:

> *"For unauthenticated or brand-new users, the Linked List is empty. When history is empty, the recommendation engine falls back gracefully to global merchandise popularity ($S_{\text{popularity}}$ Spopularity) and top ratings (S_{rating} Srating), ensuring relevant trending items are displayed without throwing null-pointer exceptions."*

Created for ShopSmart Project Viva Voce & Technical Defense.